Introduction to Controller

Traffic Light Project

Robotics & Automation pathway

Authors	: Din Sopheak Panha
Version	: 1 R 0
Promotion	: EENG4
Cursus	: EENG
Date	: 17/02/2026

A. Table of Contents

B. Table of Figures 3
C. Introduction 4
D. Project overview and Objective 5
E. System Overview 6
1. System overview 6
2. Hardware 7
3. Software 8
F. Methodology 9
1. Step-by-step process 9
2. Circuit designing in TIA Portal V18 11
3. Traffic light operation sequence 12
G. Result and Discussion 14
H. Conclusion 16
I. References 17

B. Table of Figures

Figure E-1 : System overview 6
Figure E-2 : Siemens SIMATIC S7-1200 7
Figure E-3 : AC-DC power supply 7
Figure E-4 : Input and Output panel 7
Figure E-5 : Ethernet cable 7
Figure E-6 : Power supply cable 7
Figure E-7 : Serial communication cable 7
Figure E-8 : TIA Portal V18 8
Figure F-1 : Project 1 panel 9
Figure F-2 : PLC tags folder 9
Figure F-3 : PLC tags assignment 10
Figure F-4 : Program blocks folder 10
Figure F-5 : Circuit designing panel 10
Figure F-6 : Compile, Download and Go online panel 11
Figure F-7 : Traffic light circuit designing in TIA portal V18 11
Figure F-8 : Step 1 13
Figure F-9 : Step 2 13
Figure F-10 : Step 3 13
Figure F-11 : Step 4 13
Figure G-1 : First step in real hardware 14
Figure G-2 : Second step in real hardware 14
Figure G-3 : Third step in real hardware 14
Figure G-4 : Fourth step in real hardware 14
Figure G-5 : Stop switch on 15

C. Introduction

PLC in full word calls A Programmable Logic Controller, is a ruggedized computer used for industrial automation. These controllers can automate a specific process, machine function, or even an entire production line. PLC works by receive infromation from connected sensors or input devices, process the data, and triggers outpus based on pre-programmed parameters. Depending on the inputs and outputs, a PLC can monitor and record run-time data such as machine productivity or operating temperature, automatically start and stop processes, generate alarms if a machine malfunctions, and more. Programmable Logic Controllers are a flexible and robust control solution, adaptable to almost any application [1].

Key features that set PLCs apart from industrial PCs, microcontrollers, and other industrial control solutions :

- I/O – The PLC's CPU stores and processes program data, but input and output modules connect the PLC to the rest of the machine; these I/O modules are what provide information to the CPU and trigger specific results. I/O can be either analog or digital; input devices might include sensors, switches, and meters, while outputs might include relays, lights, valves, and drives. Users can mix and match a PLC's I/O in order to get the right configuration for their application.
- Communications – In addition to input and output devices, a PLC might also need to connect with other kinds of systems; for example, users might want to export application data recorded by the PLC to a supervisory control and data acquisition (SCADA) system, which monitors multiple connected devices. PLCs offer a range of ports and communication protocols to ensure that the PLC can communicate with these other systems.
- HMI – In order to interact with the PLC in real time, users need an HMI, or Human Machine Interface. These operator interfaces can be simple displays, with a text-readout and keypad, or large touchscreen panels more similar to consumer electronics, but either way, they enable users to review and input information to the PLC in real time [1].

In this **4 hours lab**, we will use **PLC to program a traffic light operation sequence**. This report will describe the project overview, objective, methodology, and the discussion about the obtained result along with conclusion.

D. Project overview and Objective

This laboratory project involves designing, programming, downloading, and real-time testing of a basic traffic light control sequence using Siemens TIA Portal V18 software and a physical Siemens PLC. The main purpose is educational: to implement a classic operation of traffic lights and verify whether the designed logic correctly operates on actual hardware.

The objective of this lab is to design a network or circuit where :

There are two systems here. Both will have the same logic.
There are 3 outputs:

- Red light (Q0)
- Yellow light (Q1)
- Green light (Q2)

There are 2 inputs:

- I0 = Start
- I1 = Stop

When I0 is pressed:

1. Red light (Q0) turns ON for 10 seconds.
2. After 10 seconds, Red turns OFF and Yellow light (Q1) turns ON for 5 seconds.
3. After 5 seconds, Yellow turns OFF and Green light (Q2) turns ON for 7 seconds.

The same sequence is applicable to another system as well. The physical outputs may be different, but the program logic remains the same.

The system simulates the standard traffic light cycle: Red → Green → Yellow → Red. The program is developed primarily in Ladder Diagram (LAD) using timers (TON) and memory bits for state control.

After successful simulation in TIA Portal, the project emphasizes hardware-software integration:

- Downloading the program to the real PLC
- Connecting physical outputs to real hardware
- Observing real-time operation
- Analyzing and explaining the circuit behavior and step-by-step logic execution

E. System Overview

1. System overview

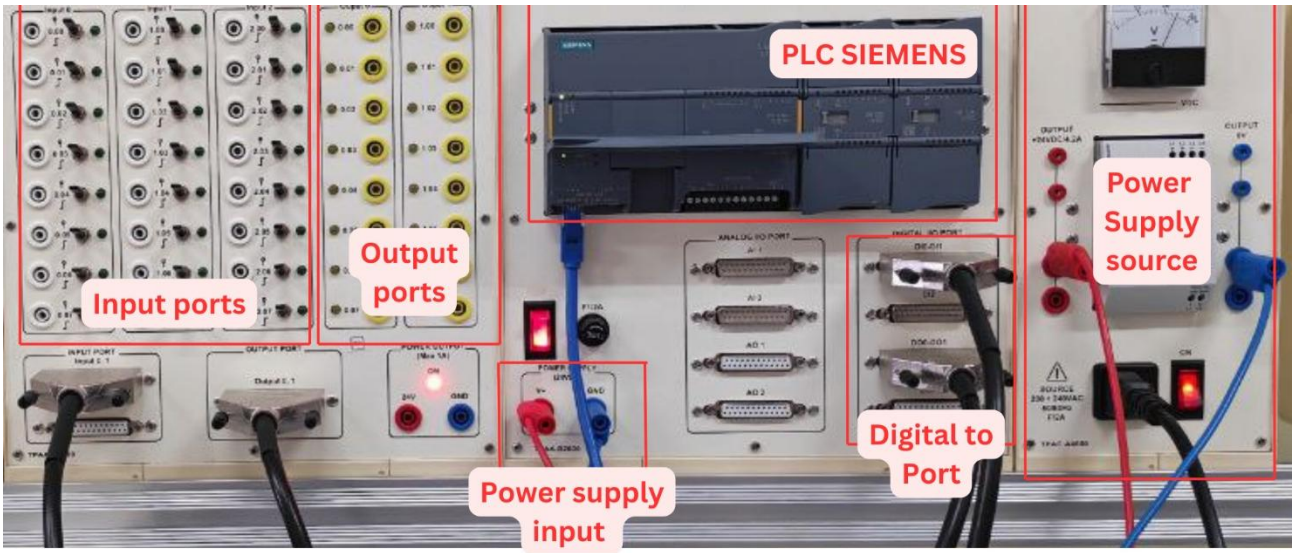

Figure E-1 : System overview

The system consists of a Siemens PLC mounted centrally on a training panel, which is surrounded by clearly labeled sections including input ports, output ports, digital I/O ports, and a power supply source. This panel is designed to simulate and test industrial automation processes.

The circuit connects the input devices and output devices to the PLC. The input ports are connected to switches that act as input signals for the PLC. When a switch on the input port is pressed, it sends an electrical signal to the PLC to indicate a command or condition. The output ports are connected to devices such as LEDs, which act as indicators. When the PLC processes the input according to the programmed logic, it sends signals to the output ports, causing the LEDs to light up to show the operation of the output devices. The PLC is connected to a laptop via an Ethernet cable, which allows for programming and monitoring of the system in real-time. The entire system is powered by a power supply source, converting the mains voltage into a stable 24V DC power required for the operation of the PLC and its connected devices.

2. Hardware

Figure E-2 : Siemens SIMATIC S7-1200

Figure E-3 : AC-DC power supply

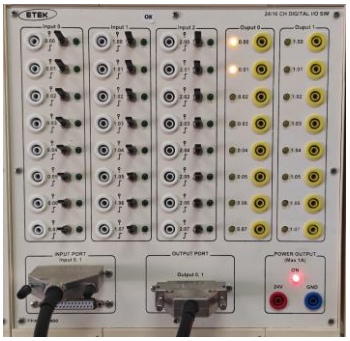

Figure E-4 : Input and Output panel

Figure E-5 : Ethernet cable

Figure E-6 : Power supply cable

Figure E-7 : Serial communication cable

The hardware that will use in this lab are :

- Siemens SIMATIC S7-1200, CPU1215FC DC/DC/DC as shown in **Figure E-2**
- AC-DC power supply, input 200-240VAC 50/60Hz to 24VDC/4.2A as shown in **Figure E-3**
- Input switch and Output panel as shown in **Figure E-4**
- Ethernet Cable as shown in **Figure E-5**
- A power supply cable that use to connect from the main power supply to the PLC system and the input/output system as shown in **Figure E-6**
- A serial communication table use to connect from the PLC system to the input/output system as shown in **Figure E-7**.

3. Software

Figure E-8 : TIA Portal V18

The system is running on a laptop with processor 12th Gen Intel(R) Core(TM) i3-1215U (1.20 GHz), RAM 8GB, Windows 11 Pro. The software we will use in this lab is TIA Portal V18 only as shown in Figure E-8.

F. Methodology

1. Step-by-step process

In this part, it is assumed that the setup of the software, setup the right PLC device and all configurations are done successfully. So, this part will focus on how to work, download and running the program in the real PLC system using TIA Portal V18.

After that, we will focus on the 'Project1' panel as shown in **Figure F-1**

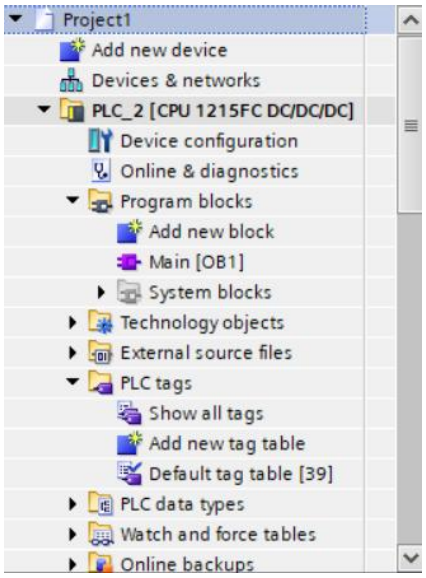

Figure F-1 : Project 1 panel

Then, we go inside the 'PLC tags' as shown in **Figure F-2** to assign the the address for the input/output port. We can double click on 'Show all tags' to assign or to review all tags that we have assigned as shown in **Figure F-3**.

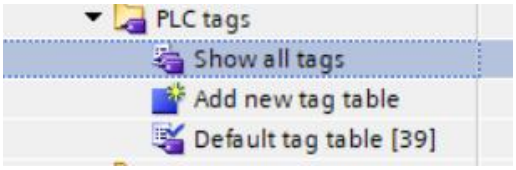

Figure F-2 : PLC tags folder

PLC tags

	Name	Tag table	Data type	Address	Retain	Acces...	Writa...	Visibl...	Comment
1	START	Default tag table	Bool	%I0.0		✓	✓	✓	
2	STOP	Default tag table	Bool	%I0.1		✓	✓	✓	
3	RED	Default tag table	Bool	%Q0.1		✓	✓	✓	
4	GREEN	Default tag table	Bool	%Q0.2		✓	✓	✓	
5	YELLOW	Default tag table	Bool	%Q0.3		✓	✓	✓	
6	RED_DONE	Default tag table	Bool	%M0.1		✓	✓	✓	
7	GREEN_DONE	Default tag table	Bool	%M0.2		✓	✓	✓	
8	YELLOW_DONE	Default tag table	Bool	%M0.3		✓	✓	✓	

Figure F-3 : PLC tags assignment

Next, we will go under 'Program blocks' folder and double tap on 'Main [OB1]' to design the circuit as shown **Figure F-4**. Then, the network designing panel will pop up as shown in **Figure F-5**.

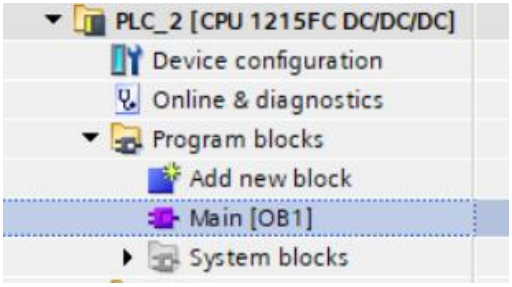

Figure F-4 : Program blocks folder

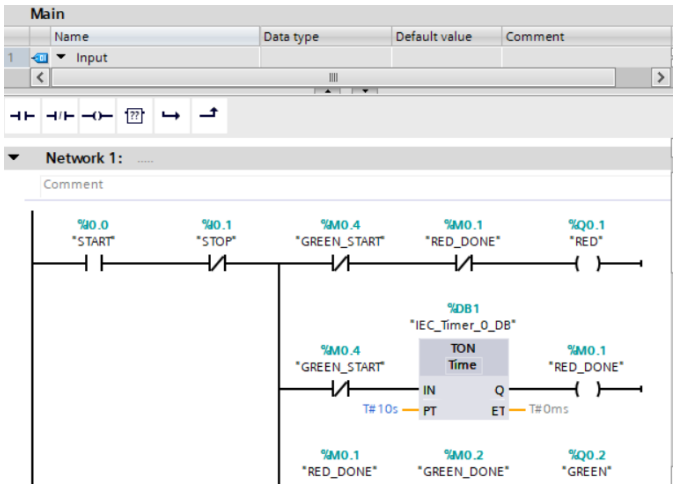

Figure F-5 : Circuit designing panel

After we design the circuit, we can do test with a real hardware. As shown in **Figure F-6,** First we select on '1' which is to compile the circuit that we have designed. If no erros, it means everything is done correctly, so we can download the code by selecting on '2' and upload to PLC. After that, select on '3' which is 'Go Online' to test the circuit with real hardware. If we want to stop it, just select on 'Go offline' back.

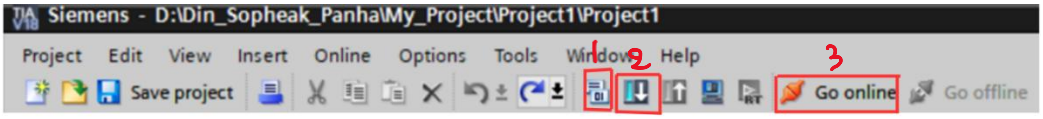

Figure F-6 : Compile, Download and Go online panel

2. Circuit designing in TIA Portal V18

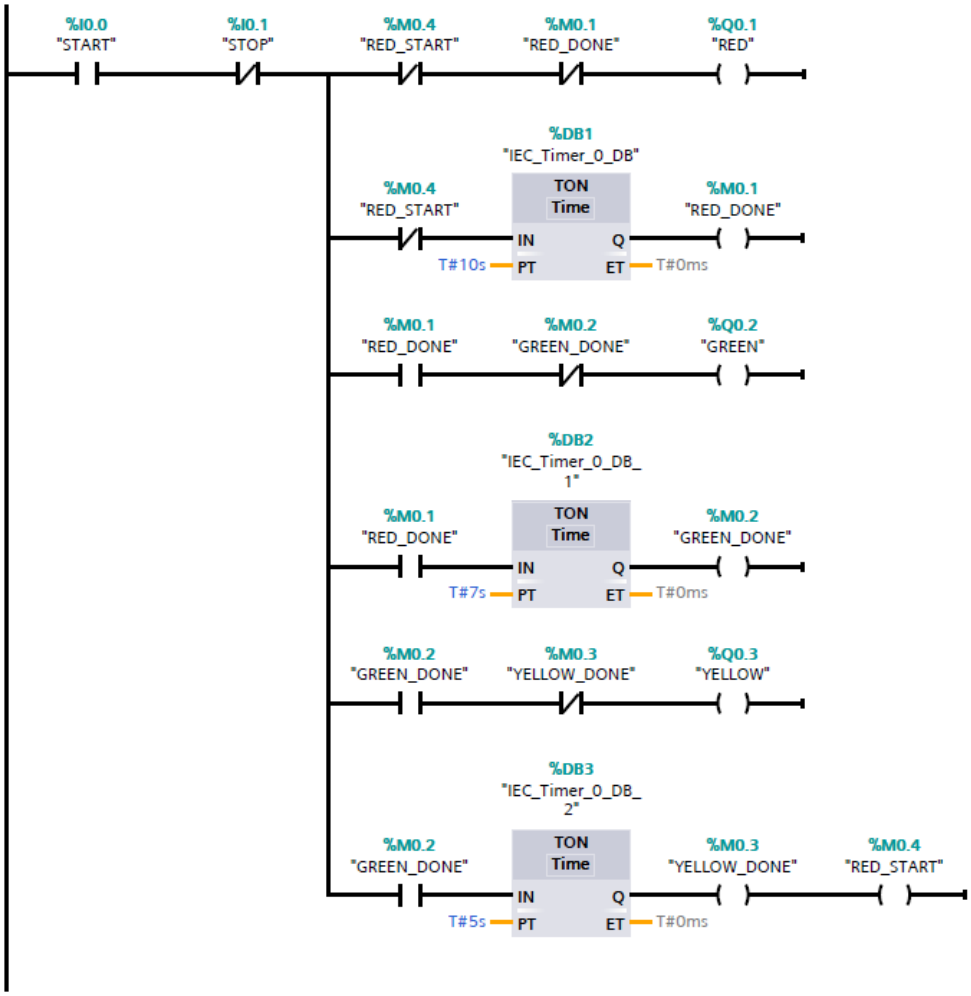

Figure F-7 : Traffic light circuit designing in TIA portal V18

The traffic light circuit designing is shown as in **Figure F-7** above.

3. Traffic light operation sequence

To easily visualize the sequence, the stop button is deleted first as shown in **Figure F-8**.

Inputs	Outputs	Memory bits	Timers
START	RED	RED_START	RED_TIMER
STOP	GREEN	RED_DONE	GREEN_TIMER
	YELLOW	GREEN_DONE	YELLOW_TIMER
		YELLOW_DONE	

Rung / Purpose	Logic summary	Description
1. Start → Red light & timer	START → RED_START → RED_DONE → RED also starts RED_TIMER	Turns RED on and begins 10-second red countdown as soon as START is pressed (and we're not already in red-done state) as shown in **Figure F-8**
2. Red timer done	RED_START → RED_TIMER.Q → RED_DONE	Sets RED_DONE latch once 10 seconds of red have passed as shown **Figure F-8**
3. Red phase output (latching)	RED_DONE → RED	RED stays on as long as red phase is not finished as shown in **Figure F-8**
4. Red done → Green light	RED_DONE → GREEN_DONE → GREEN	Turns GREEN on immediately after red finishes (and green not yet done) as shown in **Figure F-9**
5. Green timer start & done	RED_DONE → starts GREEN_TIMER GREEN_TIMER.Q → GREEN_DONE	Starts 7-second green countdown right when red ends; sets GREEN_DONE after 7 s as shown in **Figure F-10**
6. Green phase output	GREEN_DONE → GREEN	GREEN stays lit until its timer finishes as shown in **Figure F-9**
7. Green done → Yellow light	GREEN_DONE → YELLOW_DONE → YELLOW	Turns YELLOW on right after green ends as shown in **Figure F-10**
8. Yellow timer & restart	GREEN_DONE → starts YELLOW_TIMER YELLOW_TIMER.Q → YELLOW_DONE + RED_START	Runs 5-second yellow; when finished → sets YELLOW_DONE and pulses RED_START → restarts whole cycle as shown in **Figure F-11**

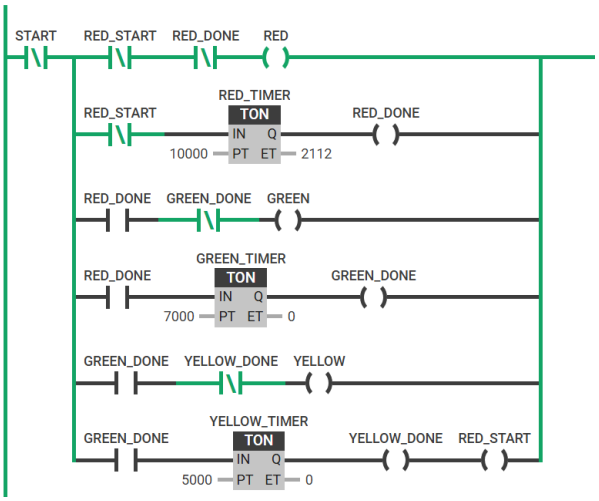

Figure F-8 : Step 1

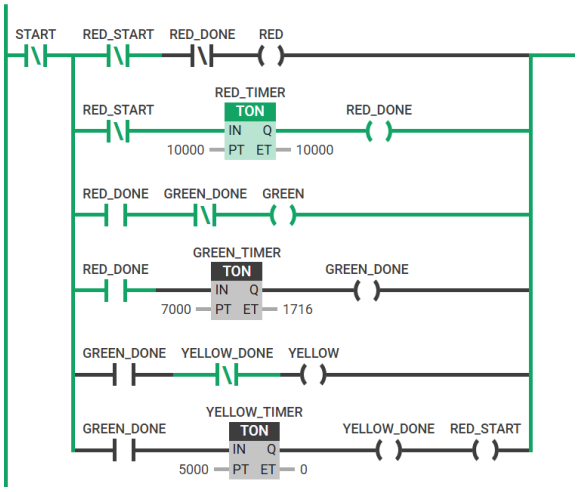

Figure F-9 : Step 2

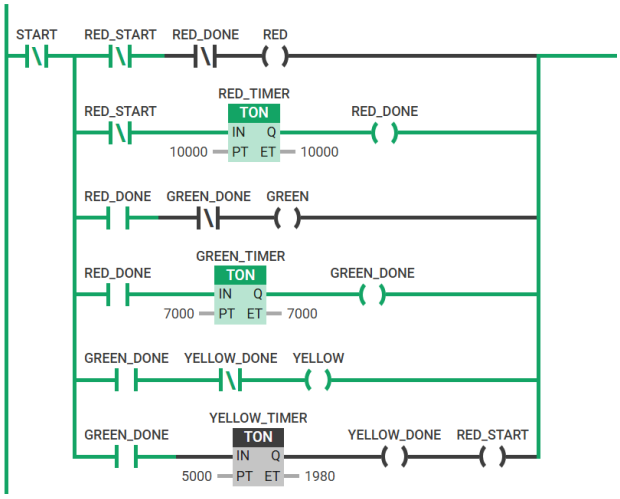

Figure F-10 : Step 3

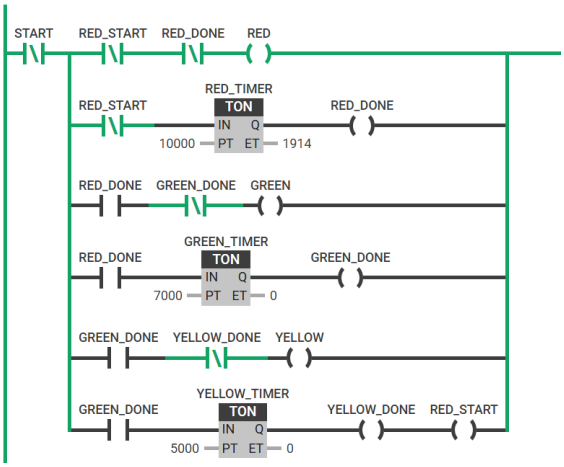

Figure F-11 : Step 4

G. Result and Discussion

The simulation result is showing in real life in the video link [2]. According to the video, we can see that when the switch 0.00 is being closed, the output 0.01 led is being on immediately as shown in **Figure G-1**.

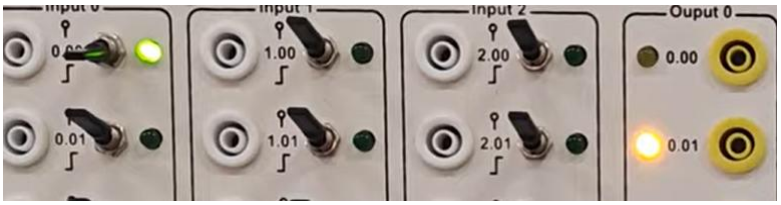

Figure G-1 : First step in real hardware

After waiting for 10 seconds, the output 0.01 is off and the ouput 0.02 led is on which is correct as shown in **Figure G-2**.

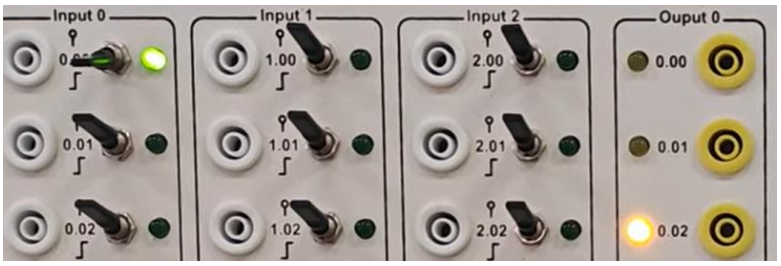

Figure G-2 : Second step in real hardware

After waiting for another 7 seconds, the output 0.02 is off and output 0.03 is on, which is correct as shown in **Figure G-3**.

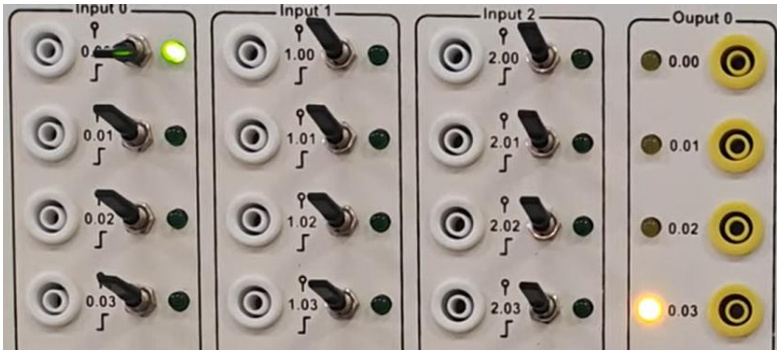

Figure G-3 : Third step in real hardware

After waiting for another 5 seconds, the loop is repeated. The output 0.01 is on again and the output 0.03 is off as shown in **Figure G-4**.

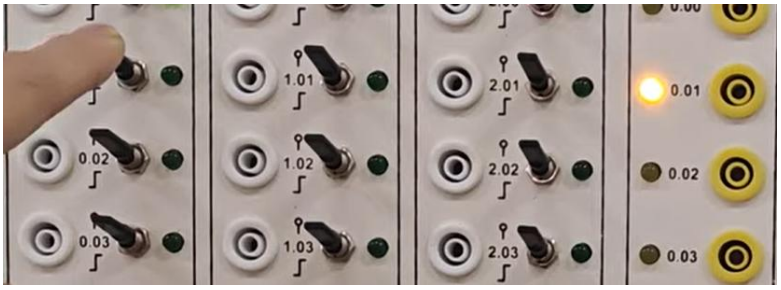

Figure G-4 : Fourth step in real hardware

If we close the 0.02 which is working as the input label 'STOP', the system got shutdown and restart from the first rung immediately which is correct as shown in **Figure G-5**.

Figure G-5 : Stop switch on

H. Conclusion

In conclusion, this laboratory project was a complete success and provided valuable hands-on experience with programmable logic controllers. We successfully designed, programmed, downloaded, and tested a basic traffic light control system using Siemens TIA Portal V18 and a real Siemens SIMATIC S7-1200 PLC. The main goal was straightforward yet highly educational: create a reliable sequence triggered by pressing the Start input (I0.0), where the red light (Q0.1) stays on for 10 seconds, then switches to yellow (Q0.2) for 5 seconds, followed by green (Q0.3) for 7 seconds, before looping back to red — and apply the identical logic to a second independent system. We also incorporated a Stop input (I0.2) to halt or reset the cycle instantly, ensuring safe control.

After verifying the logic in simulation (which already worked smoothly), the most rewarding moment came when we connected everything to the physical hardware: wiring the Start and Stop switches, linking the red, yellow, and green LEDs to the output terminals, downloading the program, going online, and watching the actual LEDs cycle precisely as intended — red for 10 seconds, yellow for 5 seconds, green for 7 seconds, and repeat — with no timing errors, no overlapping lights, and clean transitions every time. Pressing the Stop switch immediately shut down the sequence and allowed a fresh restart from red, exactly as designed.

This practical work made key PLC concepts really sink in: how TON timers count accurately in real time, how memory bits track the current phase without conflicts, why careful ladder rung ordering prevents unsafe states (like multiple lights on simultaneously), and how the transition from laptop simulation to live hardware behaves. Seeing the LEDs respond instantly to our code bridged the gap between theory and real industrial automation.

Overall, the project highlighted the power, reliability, and flexibility of PLCs — especially Siemens systems — for controlling sequential processes like traffic signals. It built strong skills in ladder diagram programming, tag assignment, compilation, downloading, online monitoring, and troubleshooting, while reinforcing why PLCs are so widely used in industry for safe, repeatable automation.

I. References

[1] PLC : https://www.unitronicsplc.com/what-is-plc-programmable-logic-controller/

[2] Testing : https://youtu.be/1IpVQlHPopc?si=B5smO_MyIfPK49V-